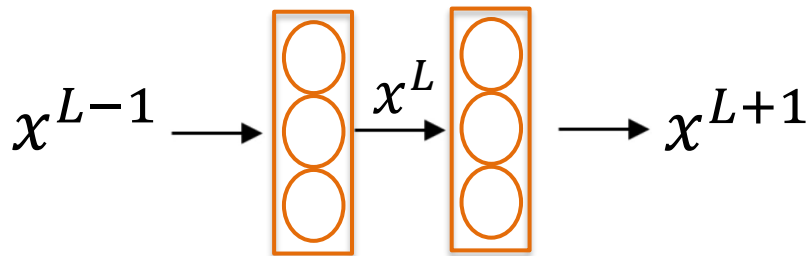
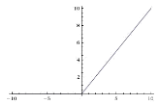


Residual Block

- Two layers



Input $\longrightarrow W^L x^{L-1} + b^L \longrightarrow x^L = f(W^L x^{L-1} + b^L) \longrightarrow$

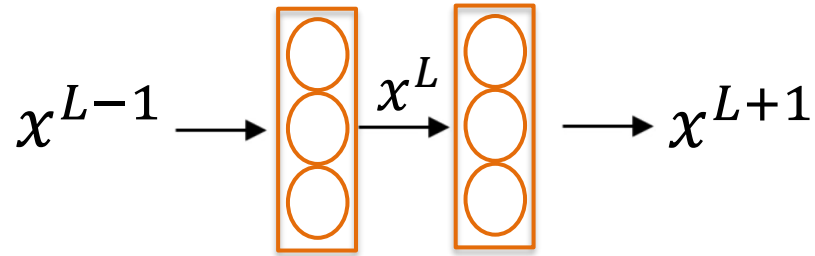
Linear  Non-linearity

The diagram shows the mathematical representation of the first layer. It starts with 'Input' followed by an orange arrow pointing to the linear transformation $W^L x^{L-1} + b^L$. Below this expression is the word 'Linear'. An orange arrow points from this expression to $x^L = f(W^L x^{L-1} + b^L)$. Below the function f is the word 'Non-linearity'. A small graph is placed between the 'Linear' and 'Non-linearity' labels, showing a coordinate system with a line passing through the origin (0,0) and the point (1,1), representing the identity function $y=x$. An orange arrow points from the final expression to the right.

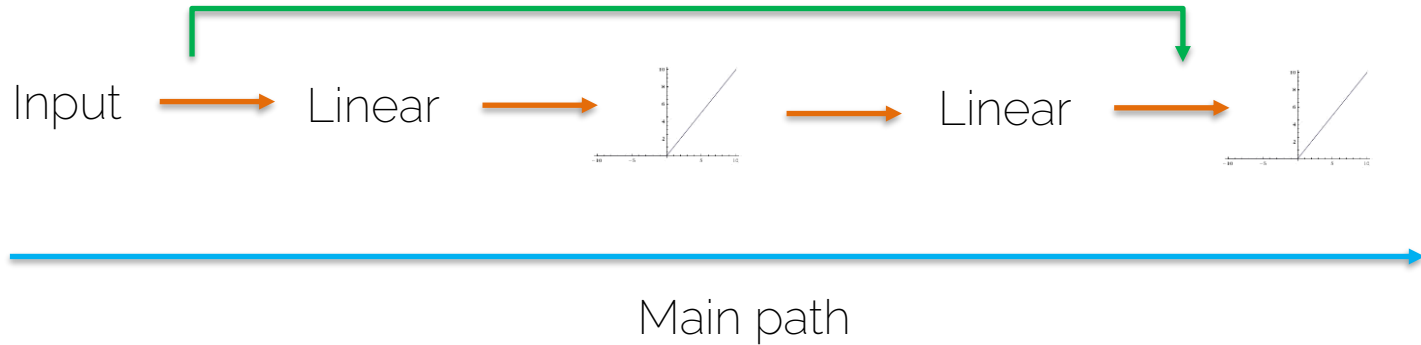
$\longrightarrow x^{L+1} = f(W^{L+1} x^L + b^{L+1})$

Residual Block

- Two layers

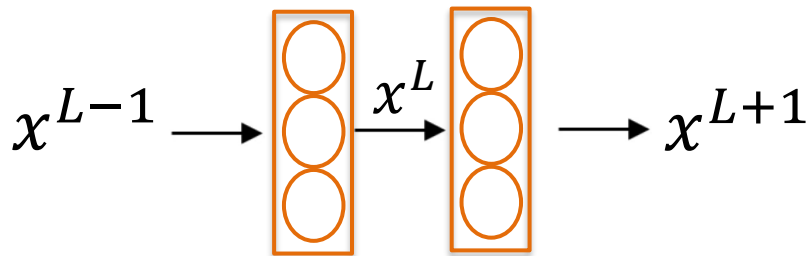


Skip connection

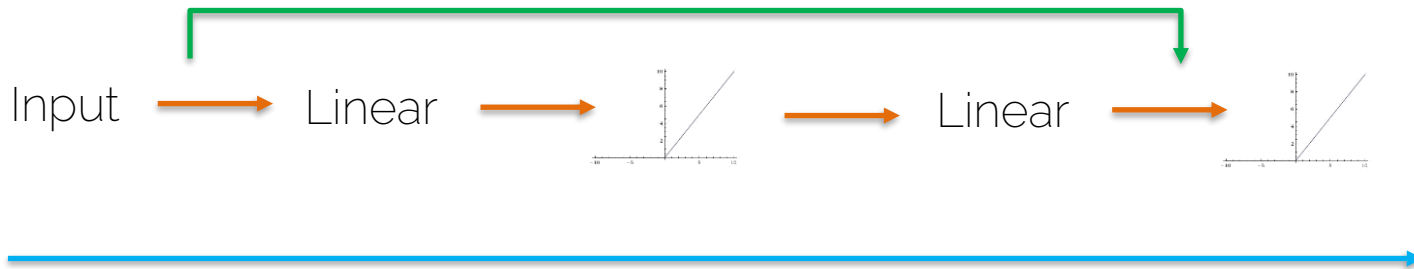


Residual Block

- Two layers



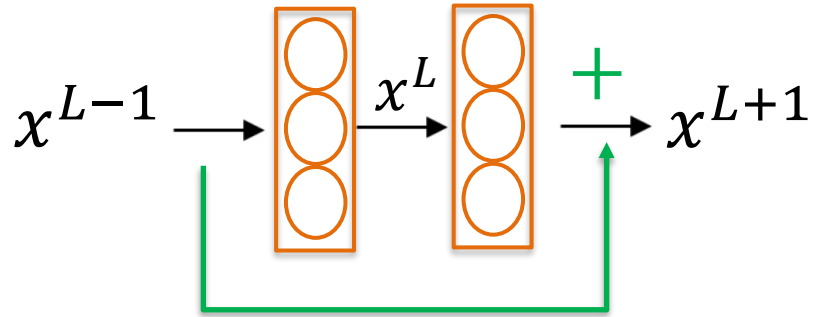
$$x^{L+1} = f(W^{L+1}x^L + b^{L+1} + x^{L-1})$$



$$x^{L+1} = f(W^{L+1}x^L + b^{L+1})$$

Residual Block

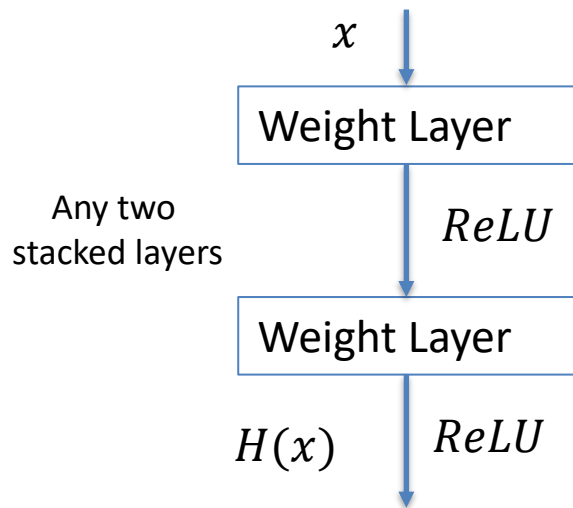
- Two layers



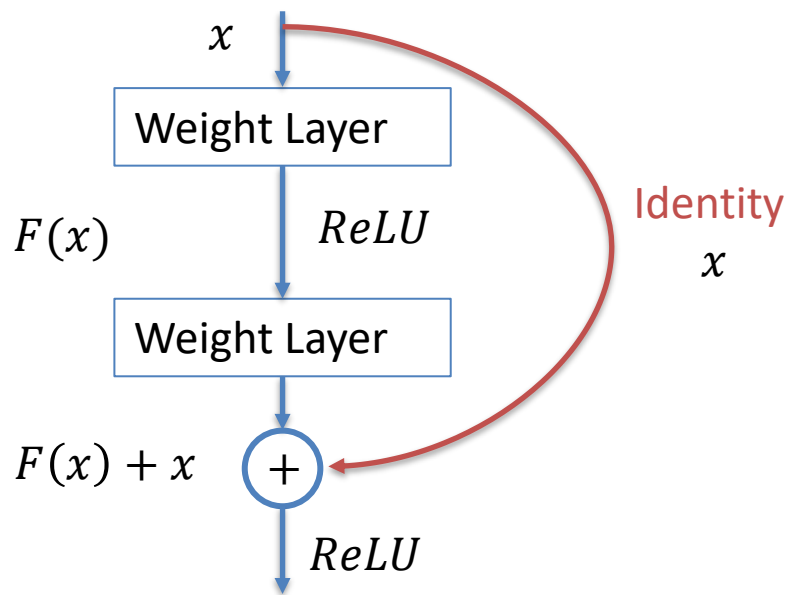
- Usually use a same convolution since we need same dimensions
- Otherwise we need to convert the dimensions with a matrix of learned weights or zero padding

ResNet Block

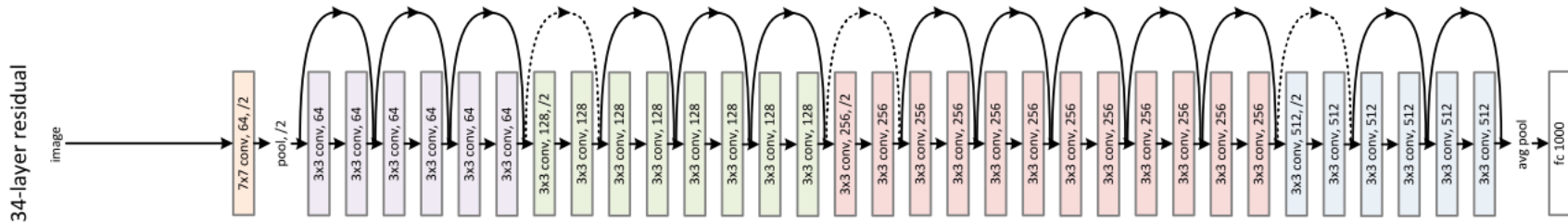
Plain Net



Residual Net



ResNet

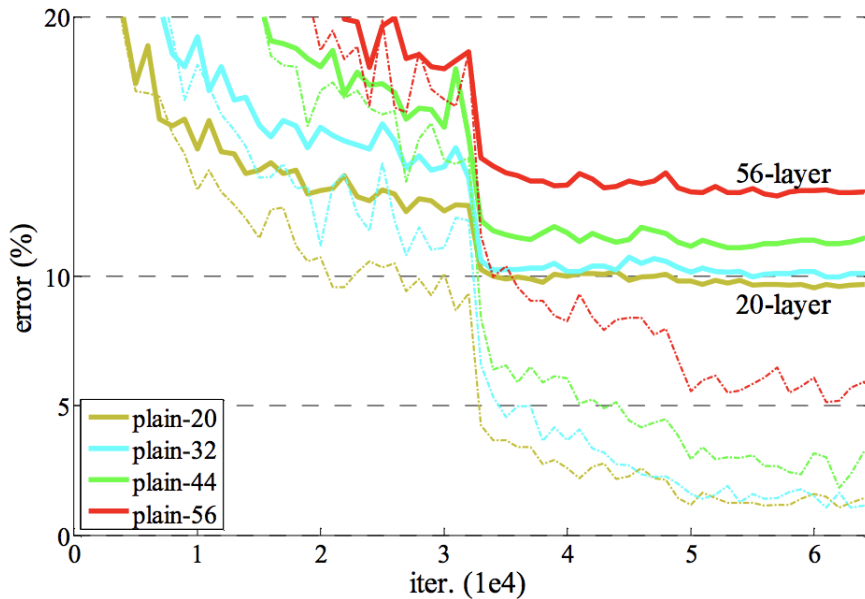


- Xavier/2 initialization
- SGD + Momentum (0.9)
- Learning rate 0.1, divided by 10 when plateau
- Mini-batch size 256
- Weight decay of $1e-5$
- No dropout

ResNet-152:
60M parameters

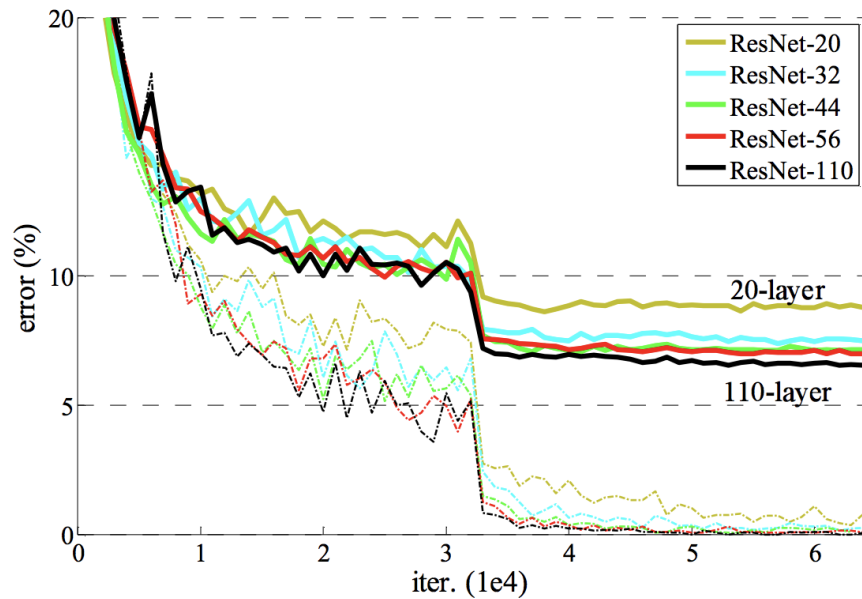
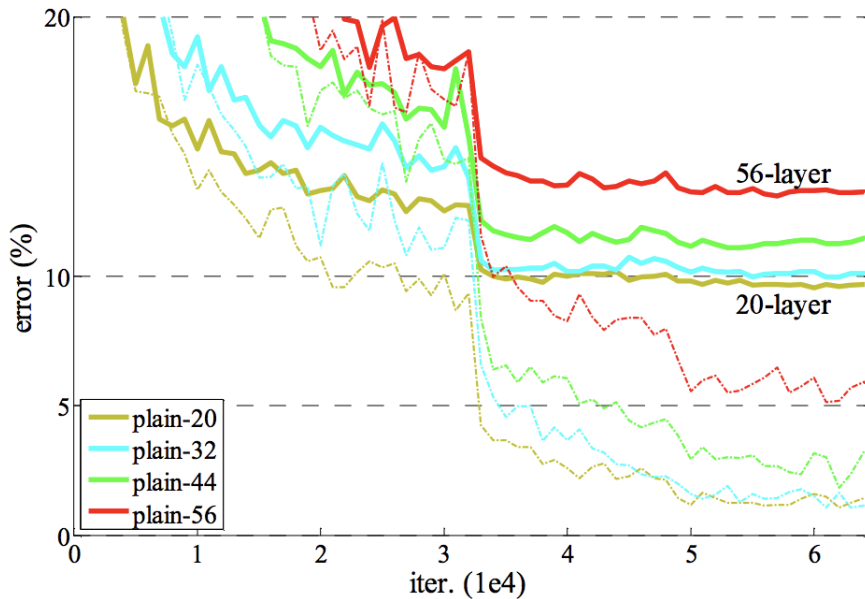
ResNet

- Without ResNet, if we make the network deeper, at some point performance starts to degrade:

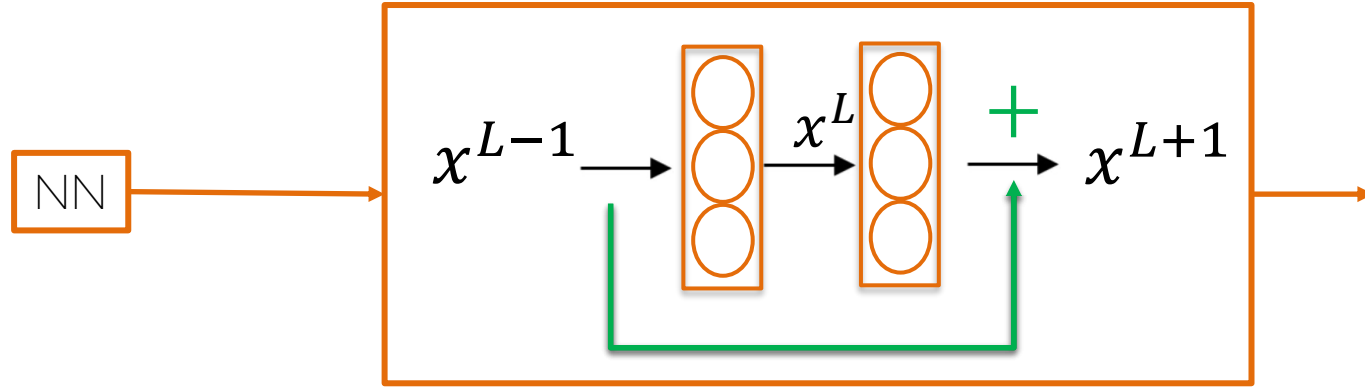


ResNet

- With Residual Blocks, performance gets better with deeper network:

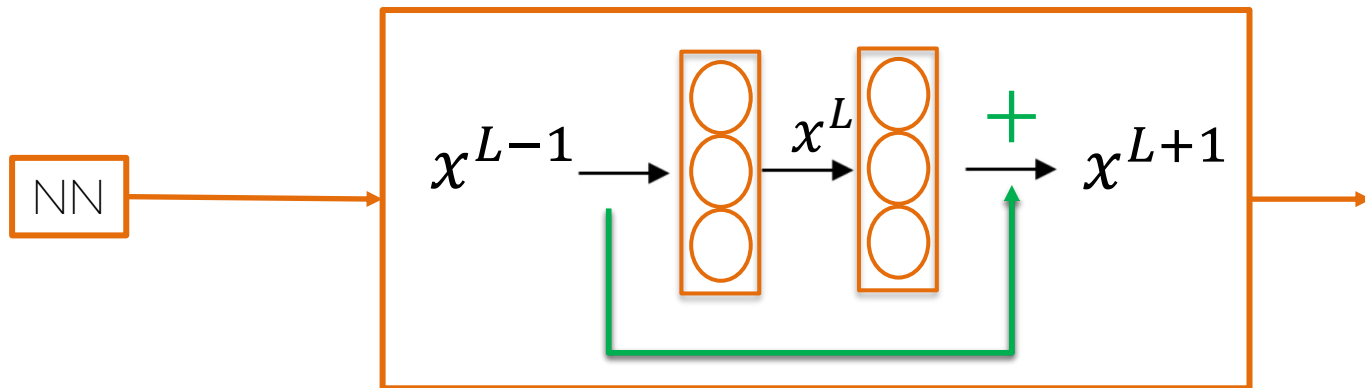


Why do ResNets Work?



- How is this block really affecting me?

Why do ResNets Work?

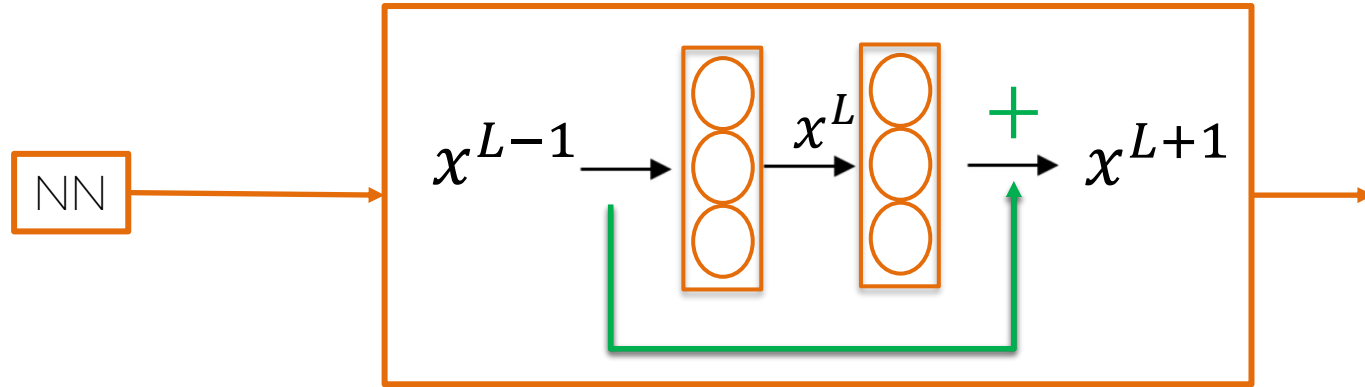


$$x^{L+1} = f(W^{L+1}x^L + b^{L+1} + x^{L-1})$$

$\sim \text{zero} \qquad \sim \text{zero}$

$$x^{L+1} = f(x^{L-1})$$

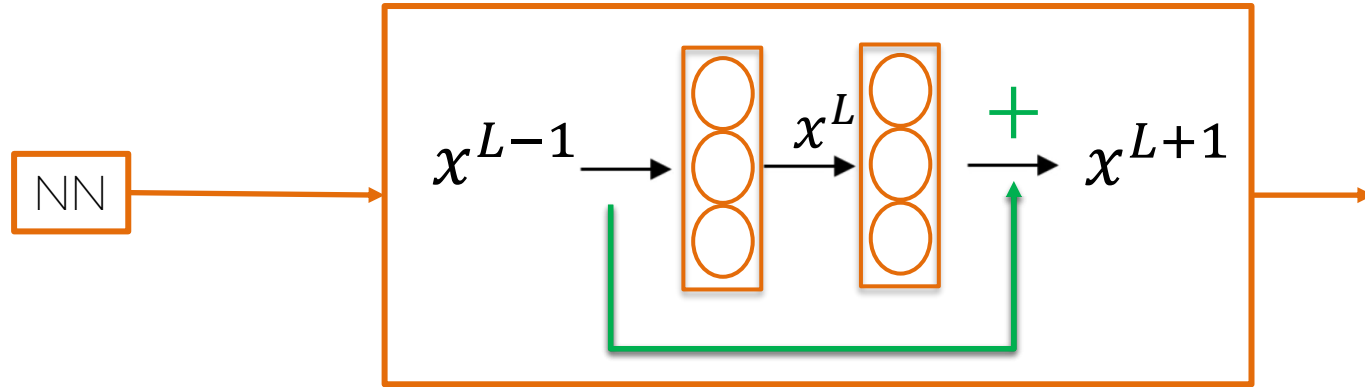
Why do ResNets Work?



- We kept the same values and added a non-linearity

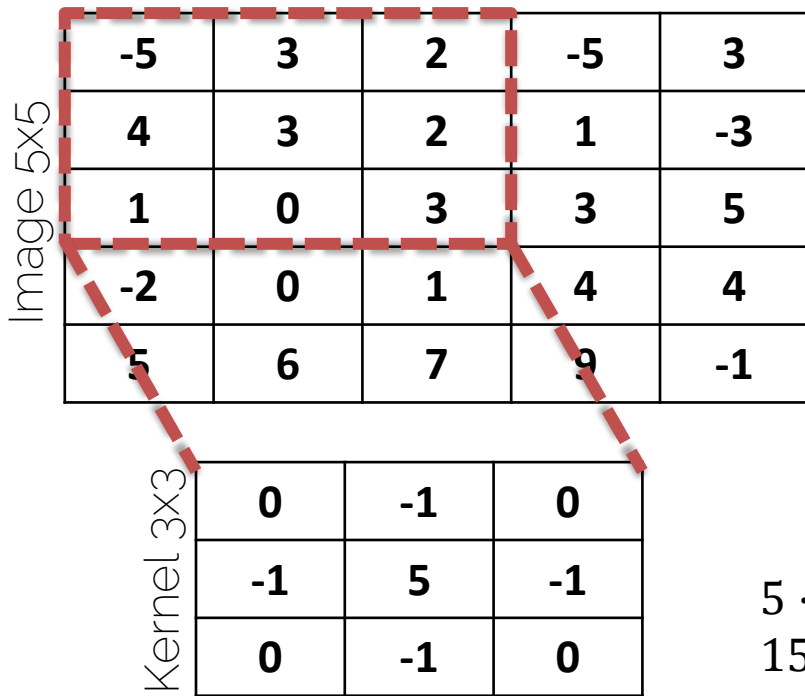
$$x^{L+1} = f(x^{L-1})$$

Why do ResNets Work?



- The identity is easy for the residual block to learn
- Guaranteed it will not hurt performance, can only improve

Recall: Convolutions on Images

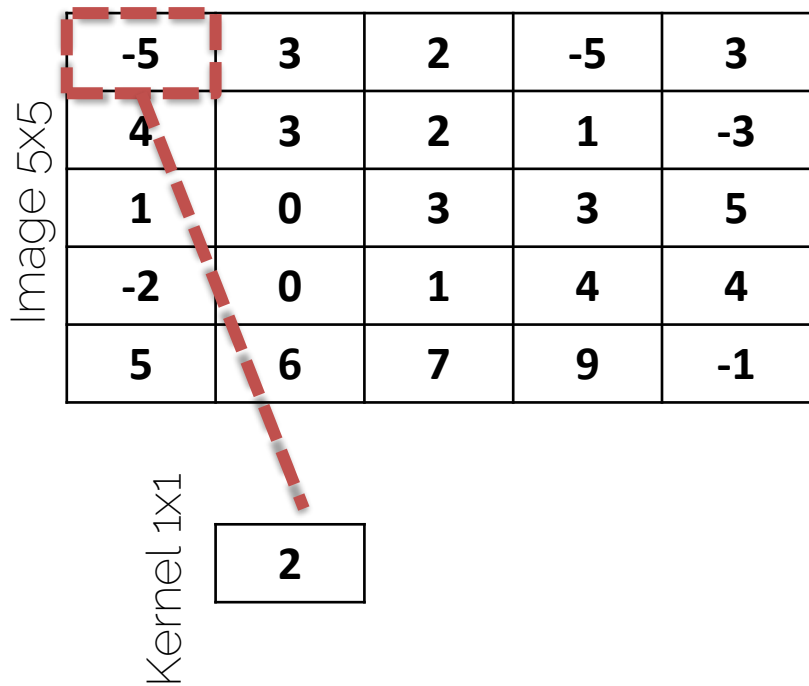


Output 3x3

6		

$$5 \cdot 3 + (-1) \cdot 3 + (-1) \cdot 2 + (-1) \cdot 0 + (-1) \cdot 4 = 15 - 9 = 6$$

1x1 Convolution



What is the output size?

1x1 Convolution

Image 5x5

-5	3	2	-5	3
4	3	2	1	-3
1	0	3	3	5
-2	0	1	4	4
5	6	7	9	-1

Kernel 1x1

2

-10				

$$-5 * 2 = -10$$

1x1 Convolution

Image 5x5

-5	3	2	-5	3
4	3	2	1	-3
1	0	3	3	5
-2	0	1	4	4
5	6	7	9	-1

Kernel 1x1

2

$$-1 * 2 = -2$$

-10	6	4	-10	6
8	6	4	2	-6
2	0	6	6	10
-4	0	2	8	8
10	12	14	18	-2

1x1 Convolution

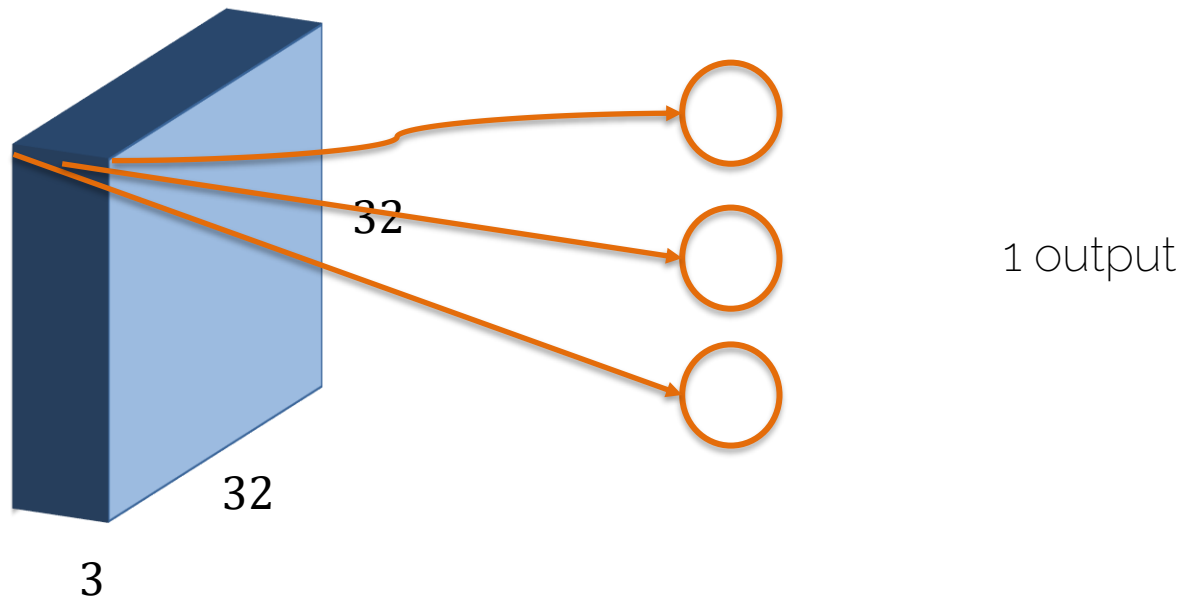
Image 5x5

-5	3	2	-5	3
4	3	2	1	-3
1	0	3	3	5
-2	0	1	4	4
5	6	7	9	-1

-10	6	4	-10	6
8	6	4	2	-6
2	0	6	6	10
-4	0	2	8	8
10	12	14	18	-2

- 1x1 kernel: keeps the dimensions and scales input

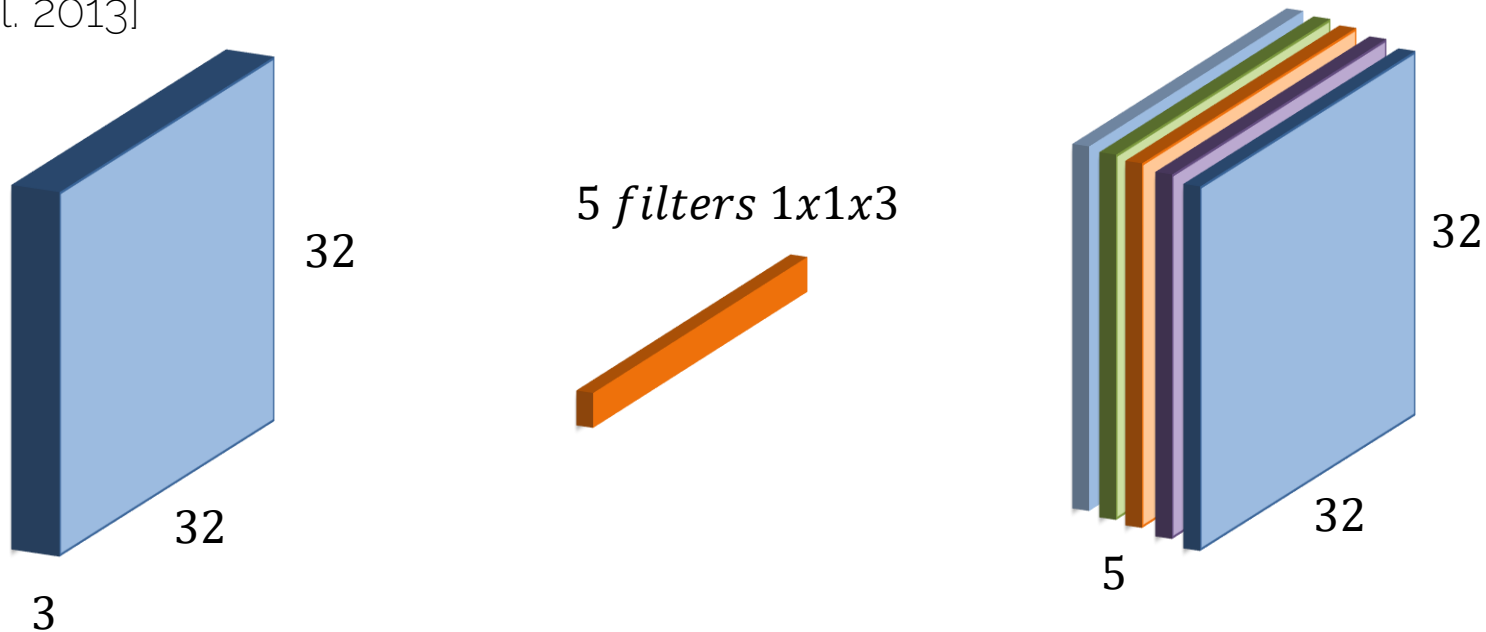
1x1 Convolution



- Same as having a 3 neuron fully connected layer

1x1 Convolution

[Li et al. 2013]



- As always we use more convolutional filters

Using 1x1 Convolutions

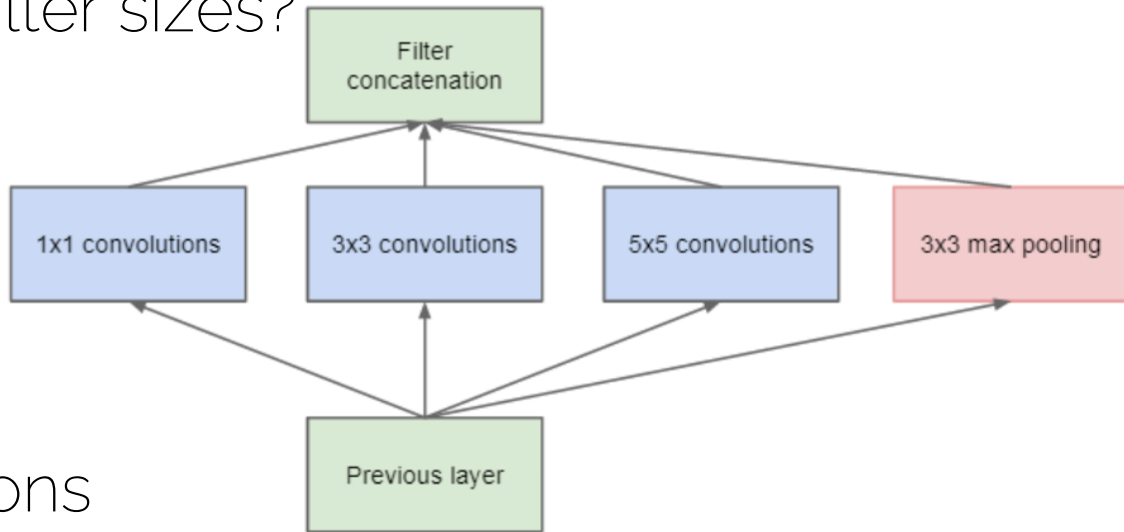
- Use it to shrink the number of channels
- Further adds a non-linearity → one can learn more complex functions



Inception Layer

- Tired of choosing filter sizes?

- Use them all!

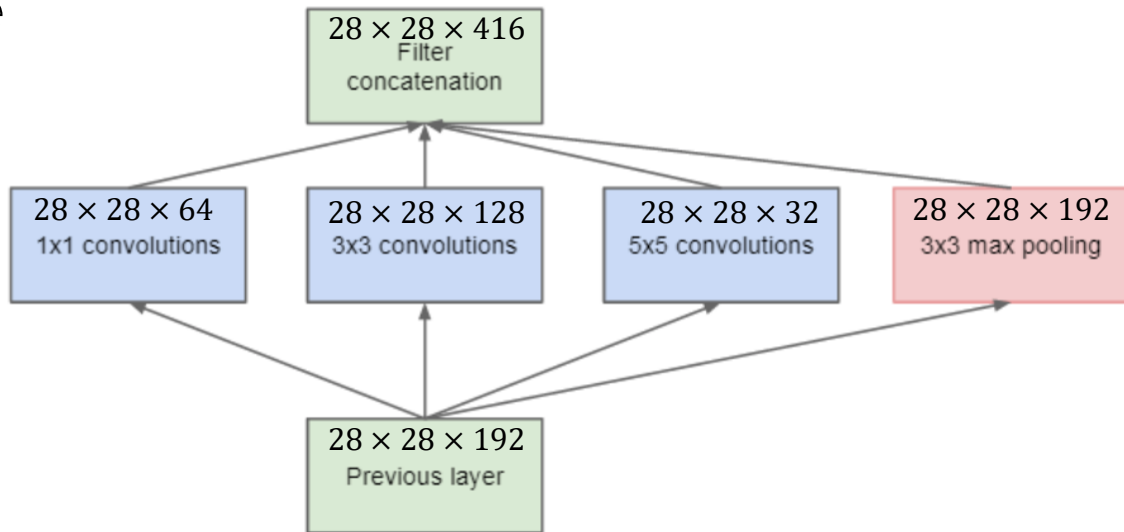


- All same convolutions

- 3x3 max pooling is with stride 1

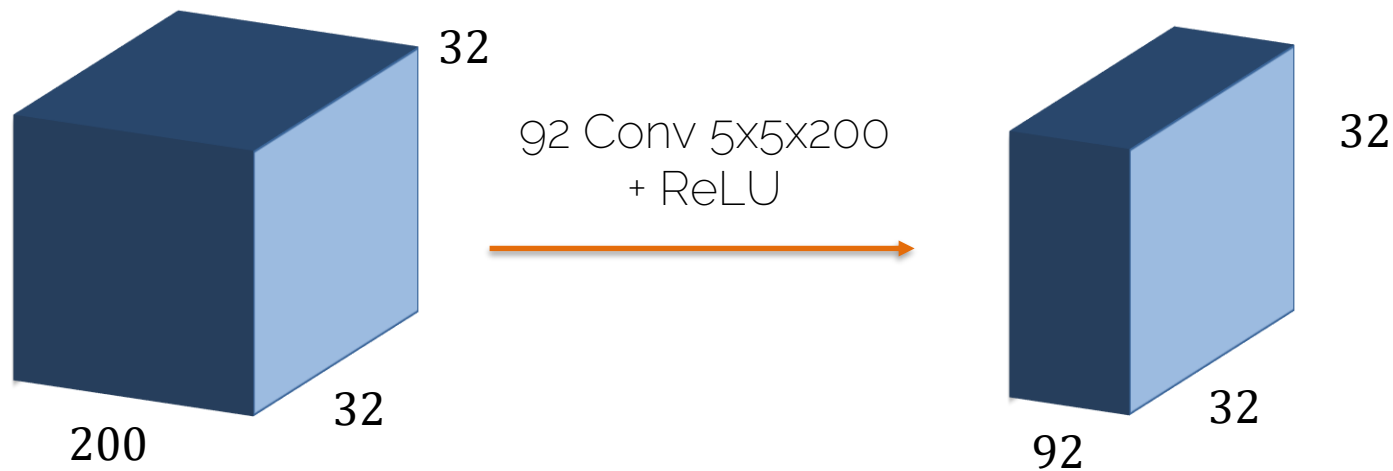
Inception Layer

- Possible size of the output



- Not sustainable!

Inception Layer: Computational Cost

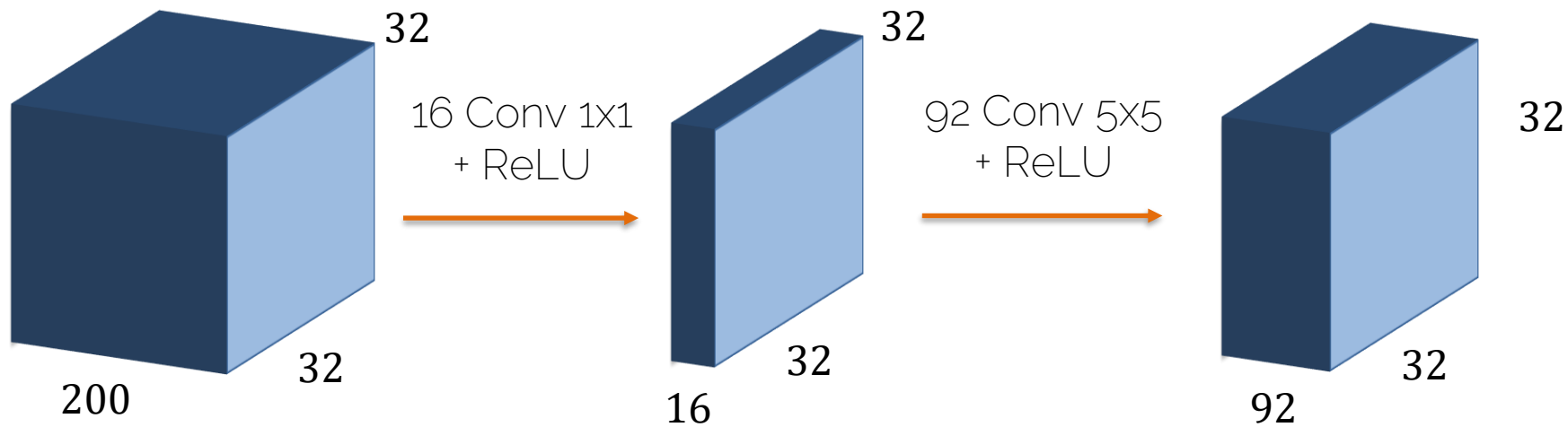


Multiplications: $5 \times 5 \times 200 \times 32 \times 32 \times 92 \sim 470$ million



1 value of the output volume

Inception Layer: Computational Cost



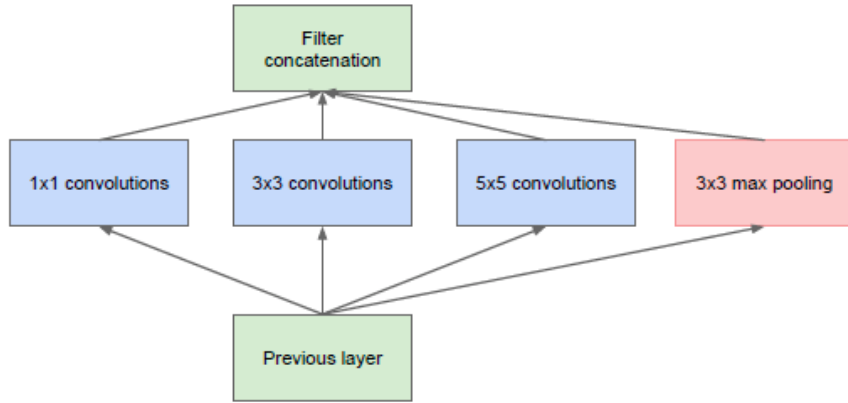
Multiplications: $1 \times 1 \times 200 \times 32 \times 32 \times 16$

$5 \times 5 \times 16 \times 32 \times 32 \times 92$

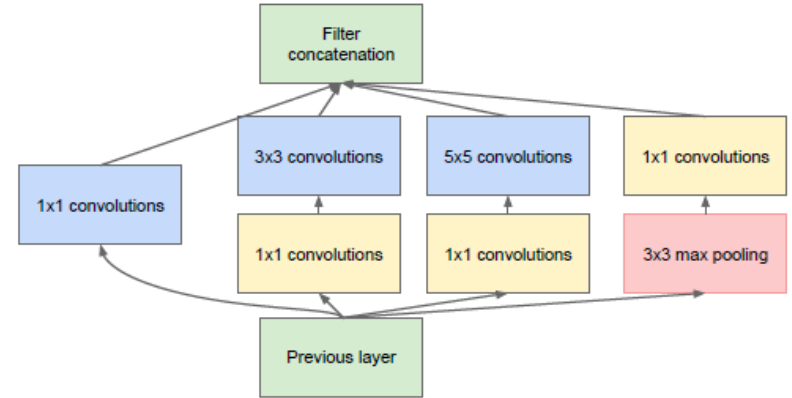
~ 40 million

Reduction of multiplications by 1/10

Inception Layer



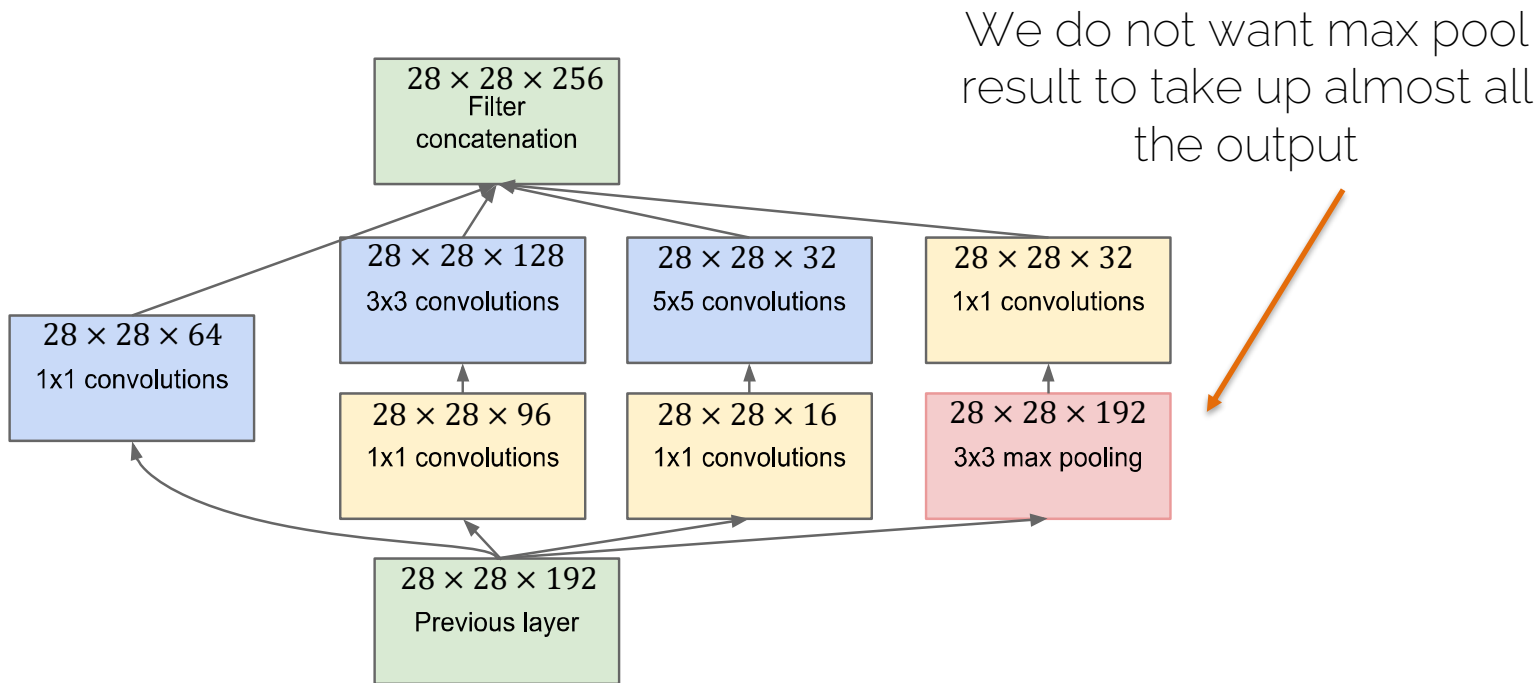
(a) Inception module, naïve version



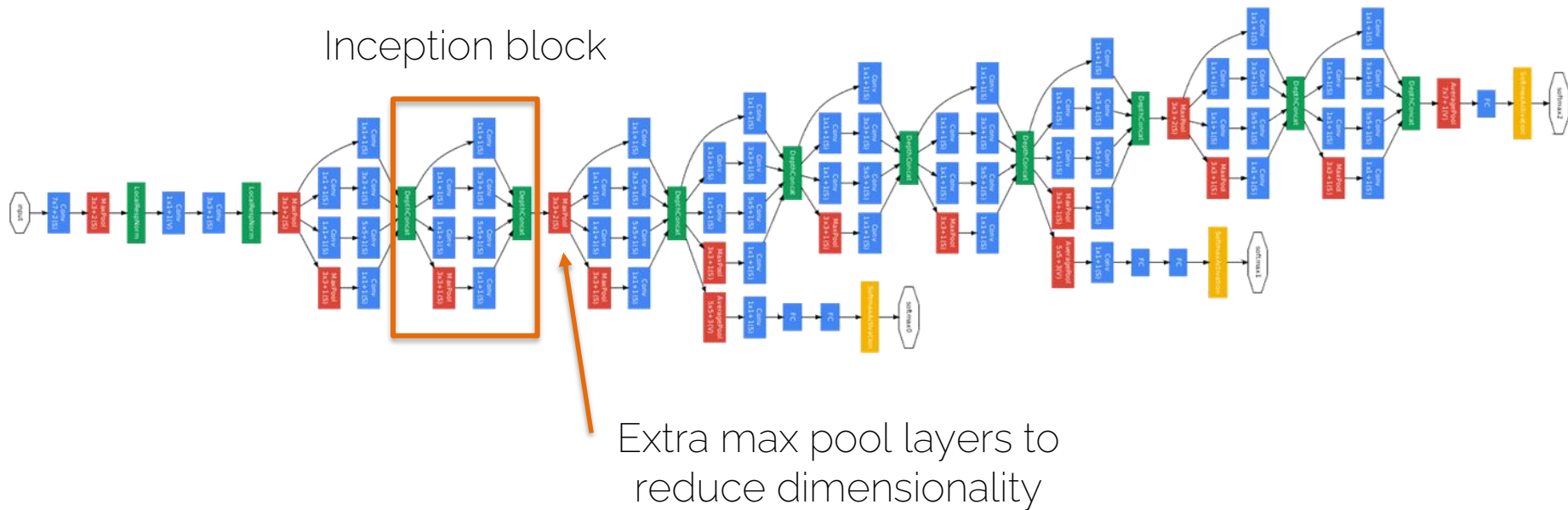
(b) Inception module with dimensionality reduction

[Szegedy et al CVPR'15] GoogLeNet

Inception Layer: Dimensions



GoogLeNet: Using the Inception Layer

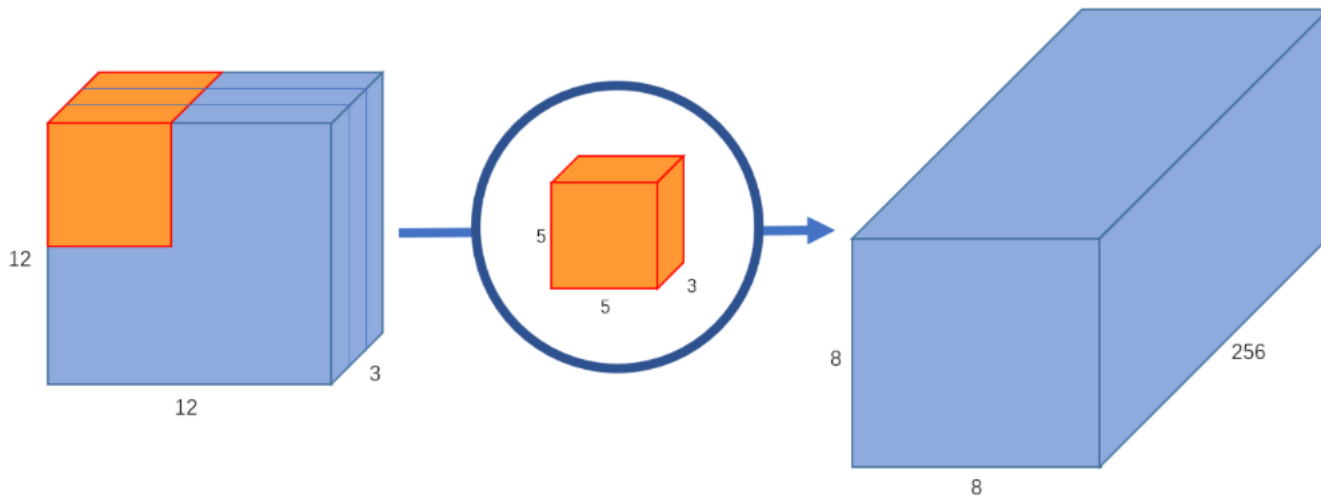


[Szegedy et al CVPR'15] GoogLeNet

Xception Net

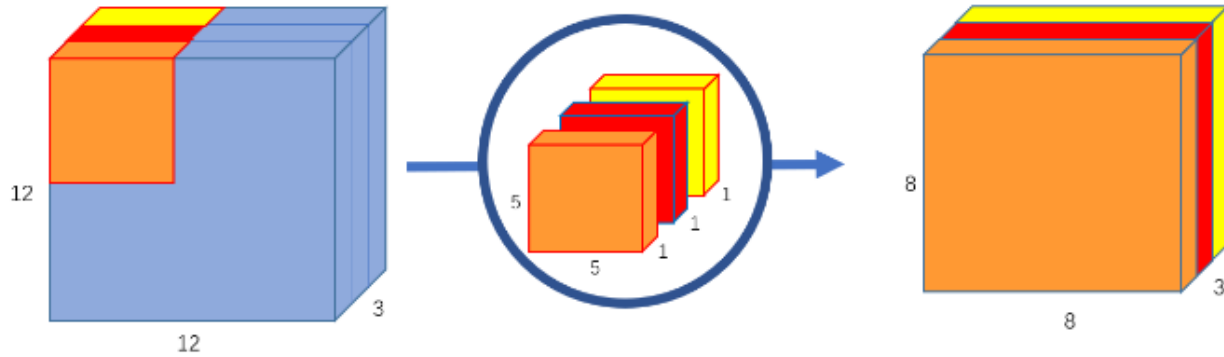
- “Extreme version of Inception”: applying (modified) **Depthwise Separable Convolutions** instead of normal convolutions
- 36 conv layers, structured into several modules with **skip connections**
- outperforms Inception Net V3

Depth-wise separable convolutions



Normal convolutions act on all channels.

Depth-wise separable convolutions

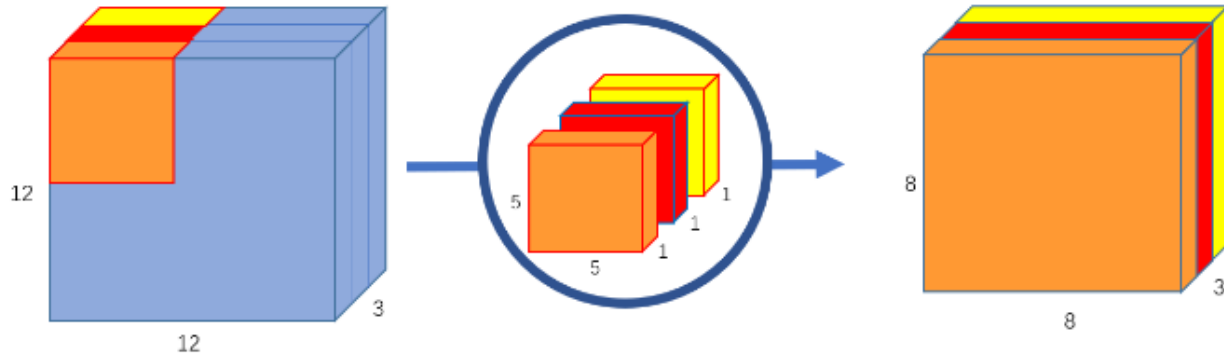


Filters are applied only at certain depths of the features. Normal convolutions have groups set to 1, the convolutions used in this image have groups set to 3.

```
class torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0, groups=3)
```

```
class torch.nn.ConvTranspose2d(in_channels, out_channels, kernel_size, stride=1, padding=0, groups=3)
```

Depth-wise separable convolutions

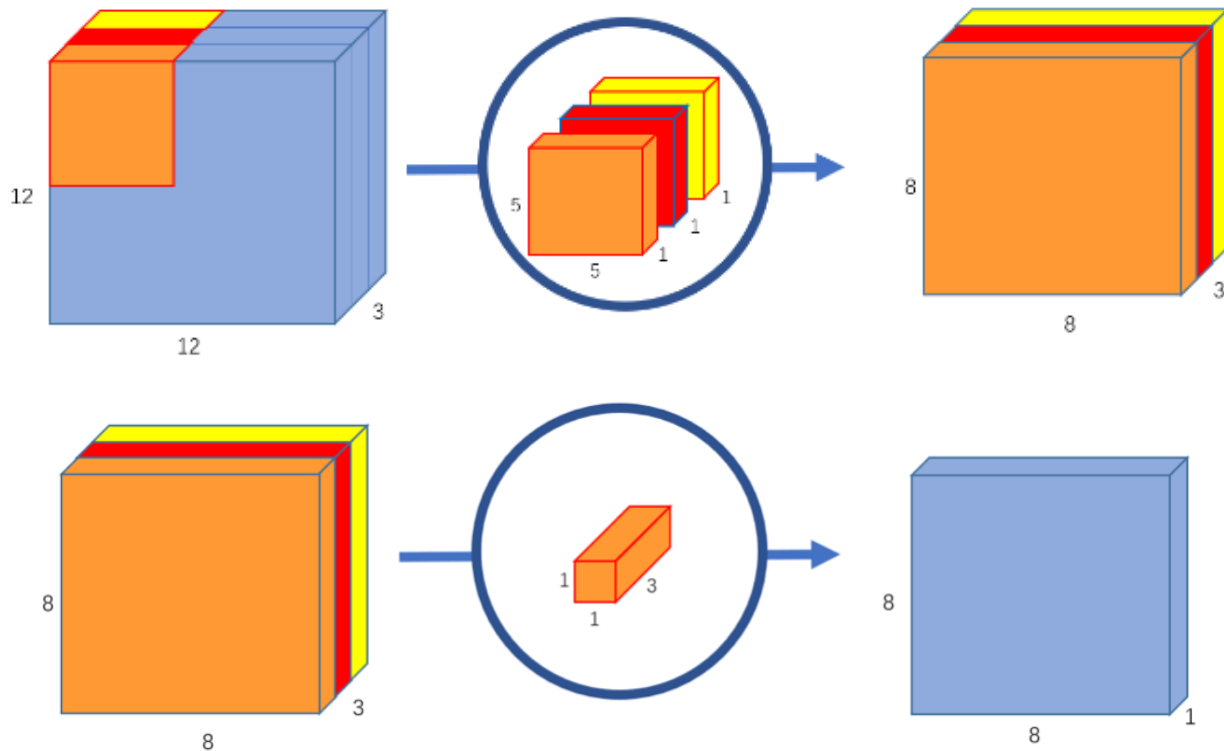


But the depth size is always the same!

`class torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0, groups=3)`

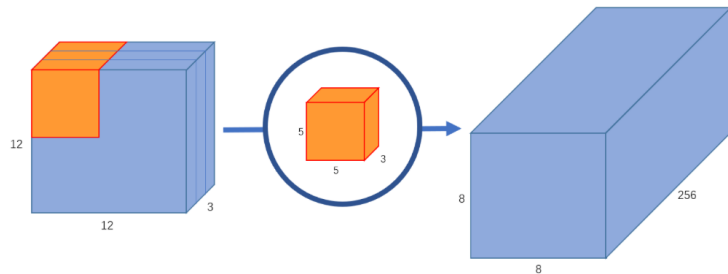
`class torch.nn.ConvTranspose2d(in_channels, out_channels, kernel_size, stride=1, padding=0, groups=3)`

Depth-wise separable convolutions



Solution:
1x1 convs!

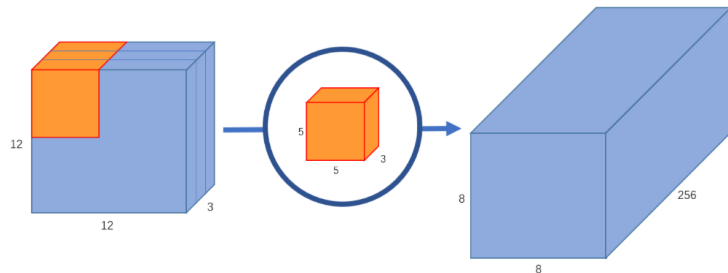
But why?



Original convolution
256 kernels of size 5x5x3

Multiplications:
 $256 \times 5 \times 5 \times 3 \times (8 \times 8 \text{ locations}) = 1,228,800$

But why?

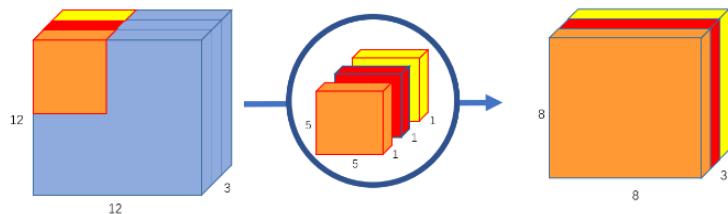


Original convolution

256 kernels of size 5x5x3

Multiplications:

$$256 \times 5 \times 5 \times 3 \times (8 \times 8 \text{ locations}) = 1,228,800$$

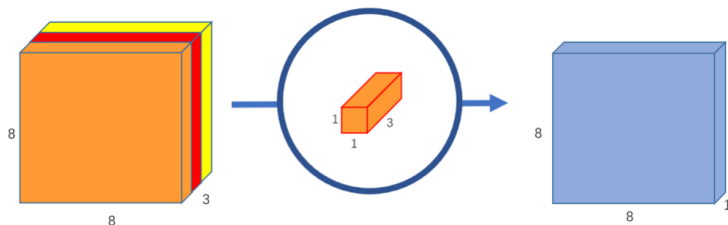


Depth-wise convolution

3 kernels of size 5x5x1

Multiplications:

$$5 \times 5 \times 3 \times (8 \times 8 \text{ locations}) = 4,800$$



1x1 convolution

256 kernels of size 1x1x3

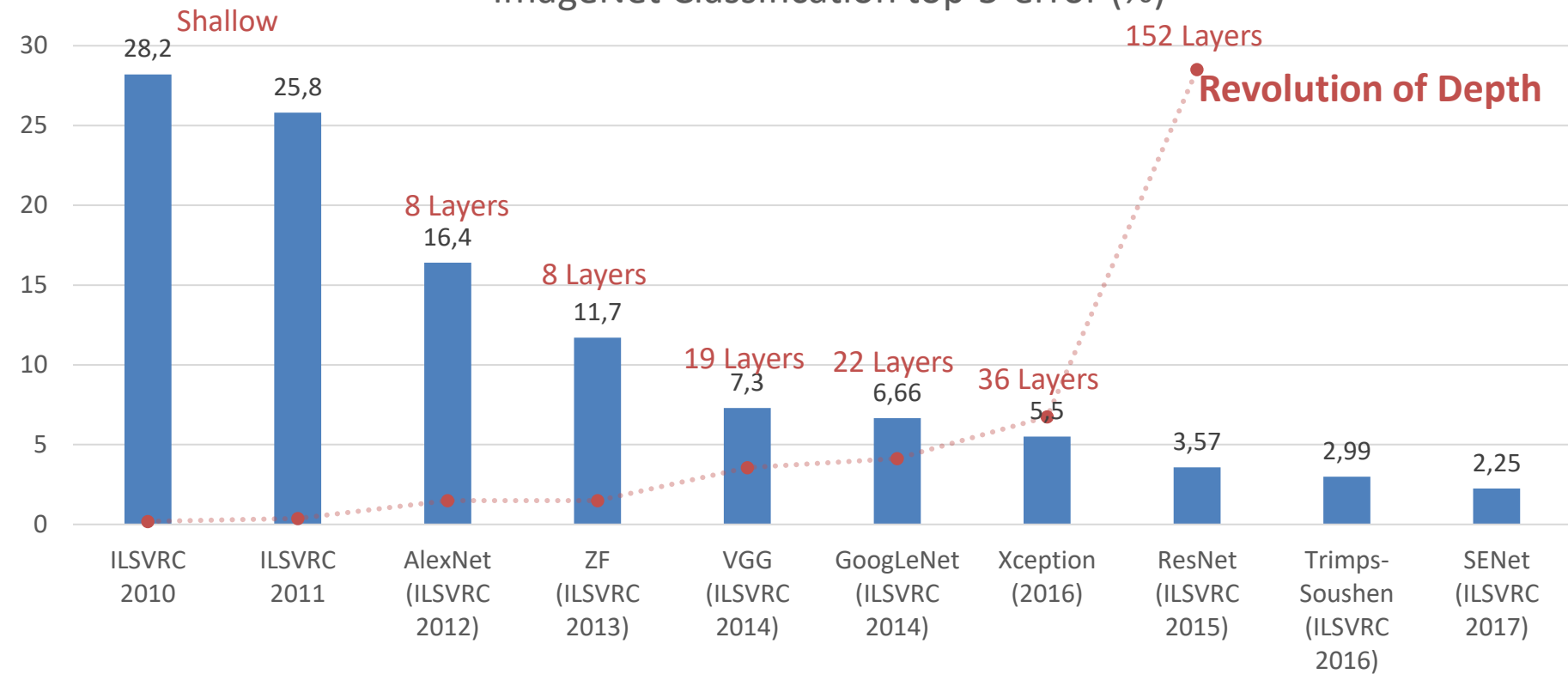
Multiplications:

$$256 \times 1 \times 1 \times 3 \times (8 \times 8 \text{ locations}) = 49,152$$

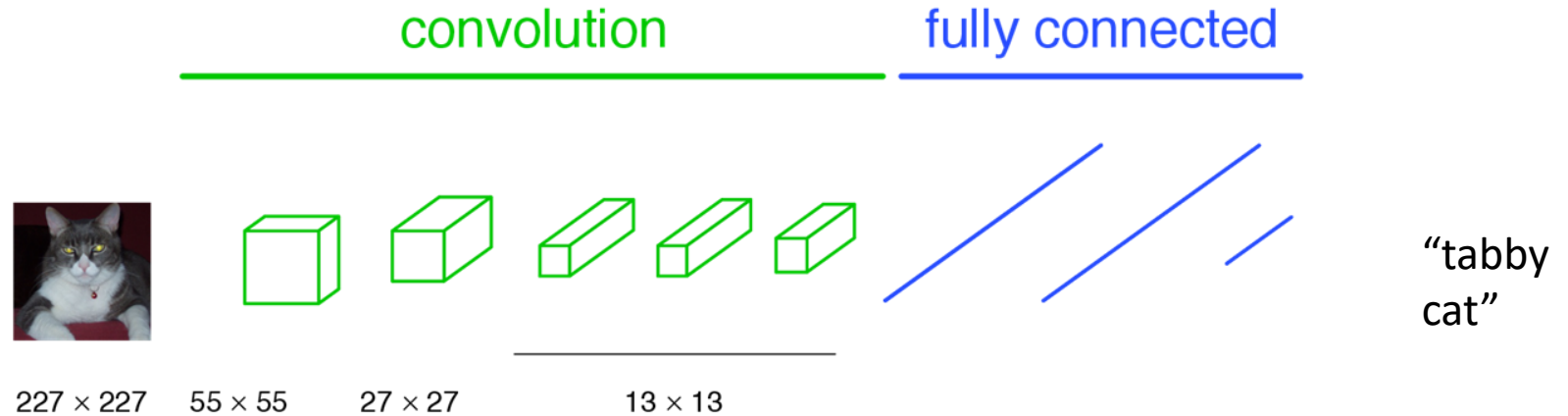
Less
computation!

ImageNet Benchmark

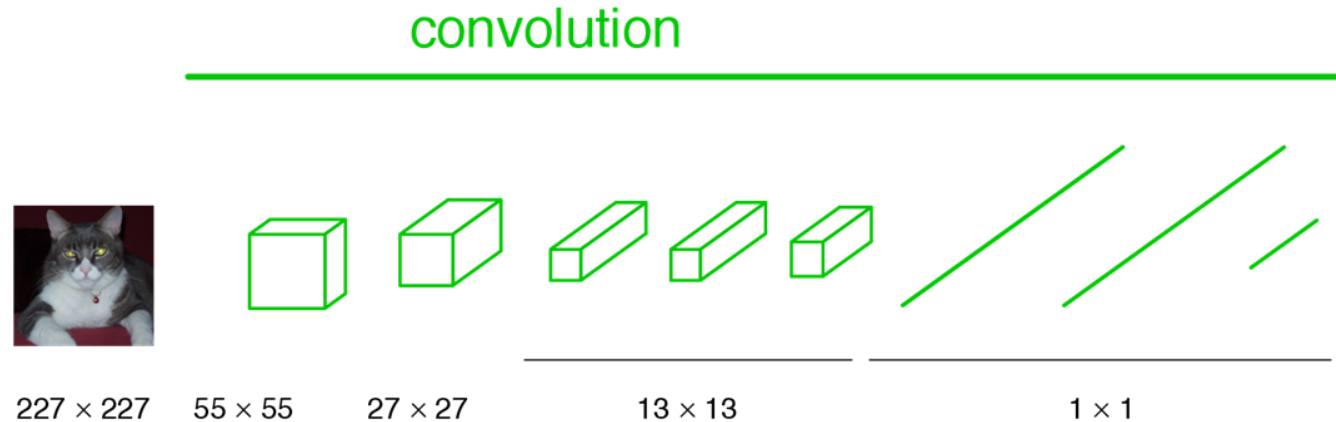
ImageNet Classification top-5-error (%)



Classification Network



FCN: Becoming Fully Convolutional



Convert fully connected layers to convolutional layers!

FCN: Becoming Fully Convolutional



$H \times W$

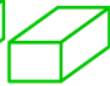
convolution



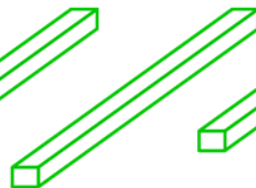
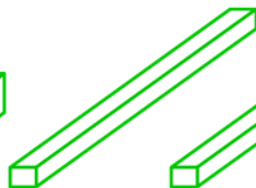
$H/4 \times W/4$



$H/8 \times W/8$



$H/16 \times W/16$



$H/32 \times W/32$

FCN: Upsampling Output

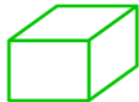


$H \times W$

convolution



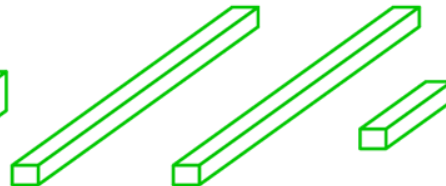
$H/4 \times W/4$



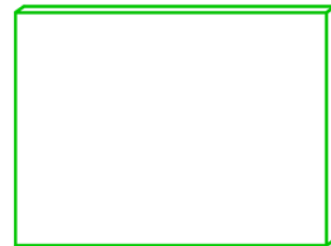
$H/8 \times W/8$



$H/16 \times W/16$

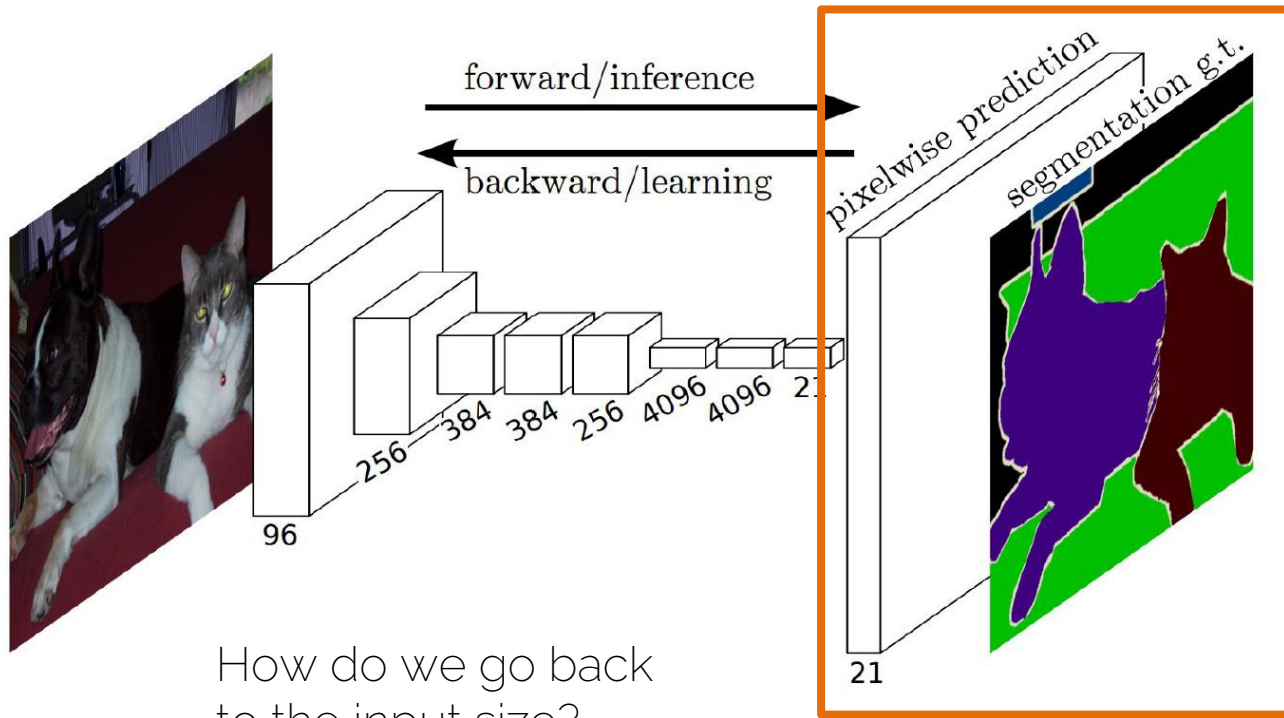


$H/32 \times W/32$



$H \times W$

Semantic Segmentation (FCN)

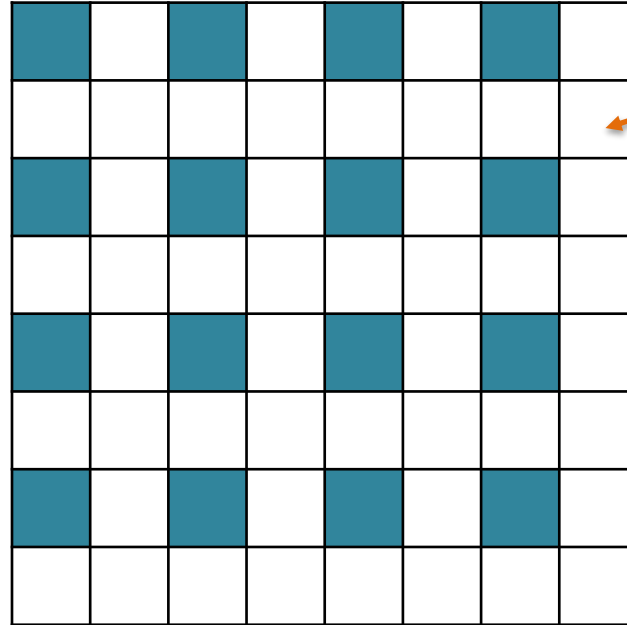
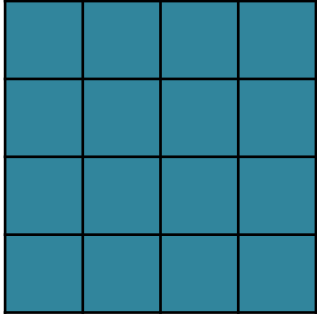


How do we go back
to the input size?

[Long and Shelhamer. 15] FCN

Types of Upsampling

- 1. Interpolation



?

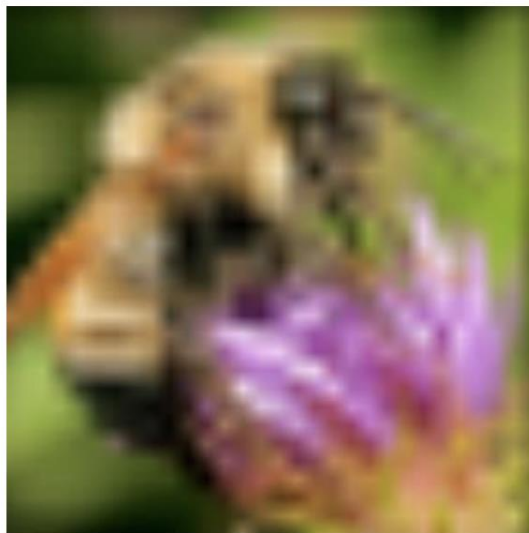
Types of Upsampling

- 1. Interpolation

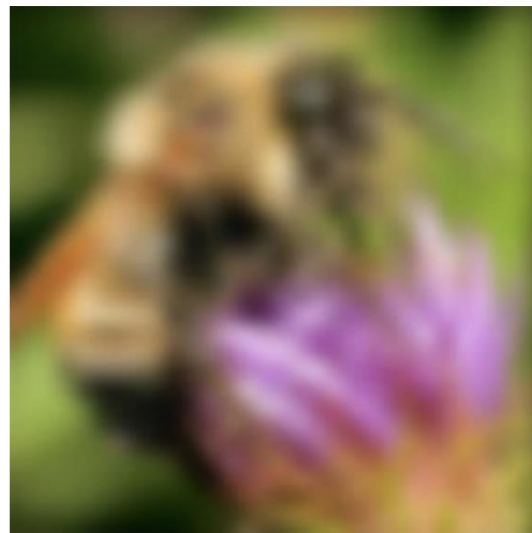
Original image  x 10



Nearest neighbor interpolation



Bilinear interpolation



Bicubic interpolation

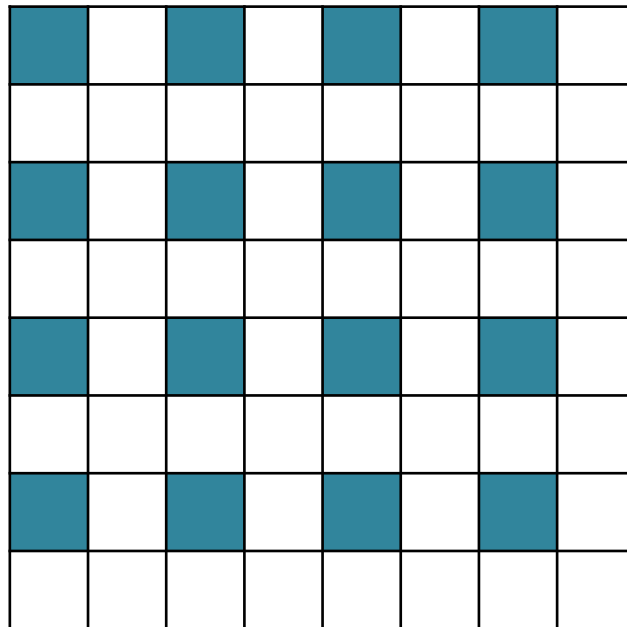
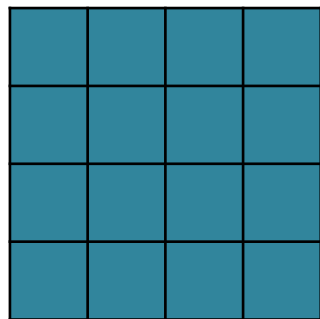
Types of Upsampling

- 1. Interpolation

 Few artifacts

Types of Upsampling

- 2. Transposed conv



+ CONVS

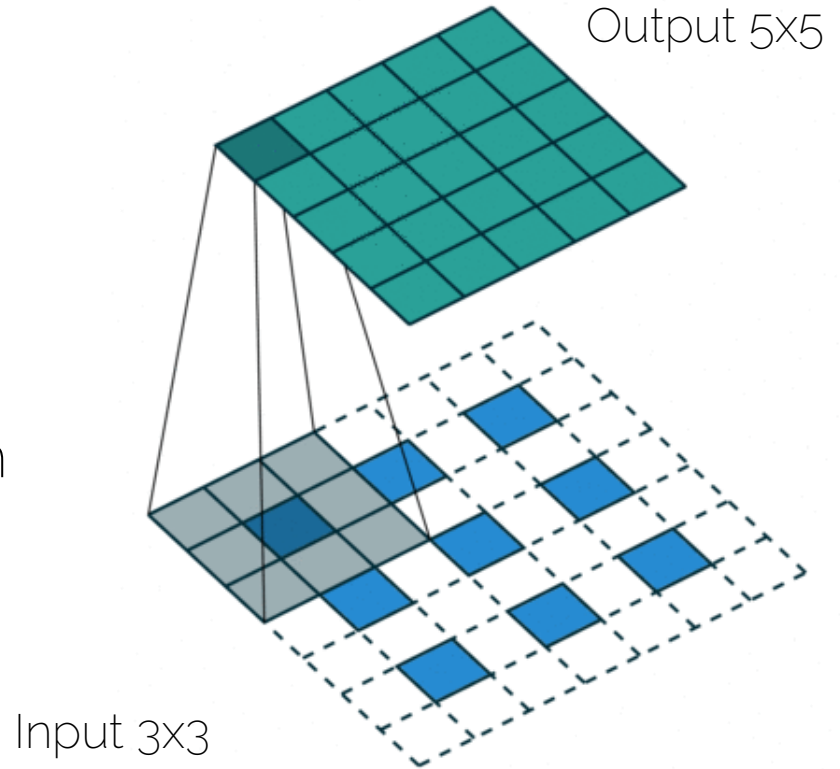


efficient

[A. Dosovitskiy, TPAMI 2017] "Learning to Generate Chairs, Tables and Cars with Convolutional Networks"

Types of Upsampling

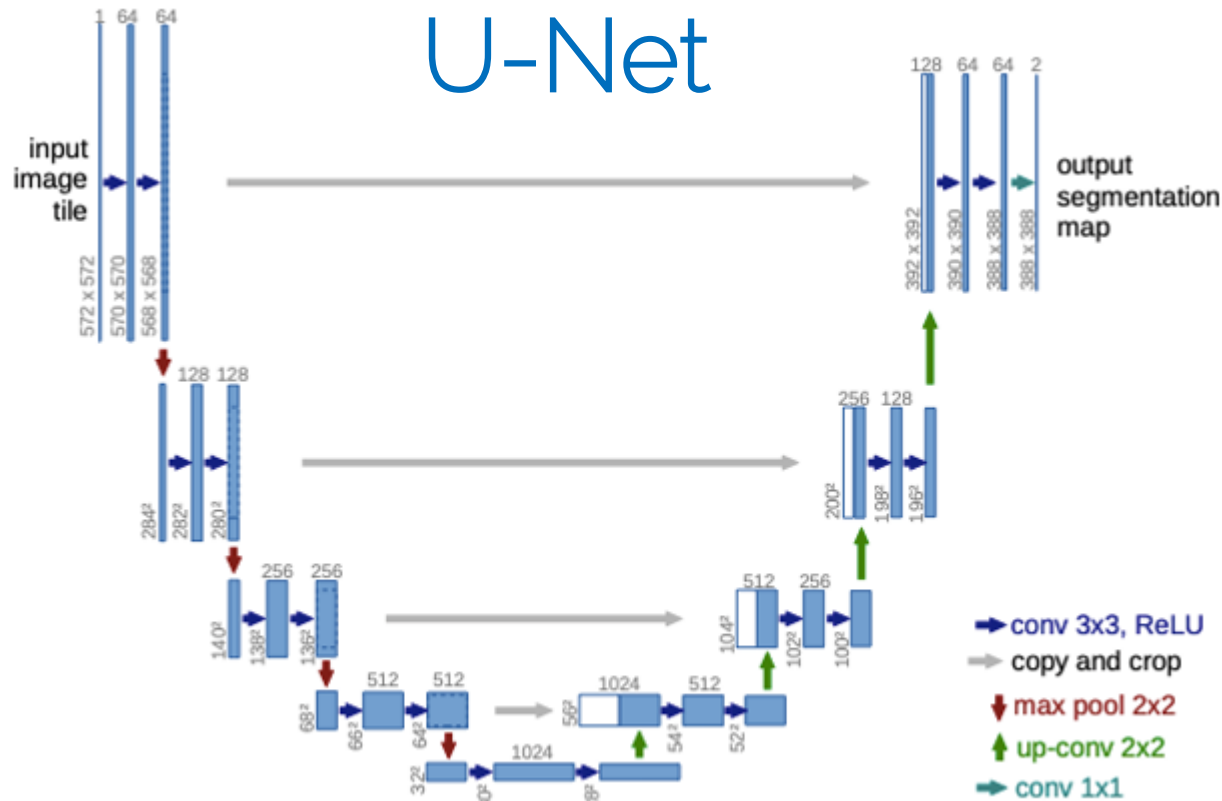
- 2. Transposed convolution
 - Unpooling
 - Convolution filter (learned)
 - Also called up-convolution (**never** deconvolution)



Refined Outputs

- If one does a cascade of unpooling + conv operations, we get to the encoder-decoder architecture
- Even more refined: Autoencoders with skip connections (aka U-Net)

U-Net



U-Net architecture: Each blue box is a multichannel feature map. Number of channels denoted at the top of the box. Dimensions at the top of the box. White boxes are the copied feature maps.

U-Net: Encoder

Left side: **Contraction Path** (Encoder)

- Captures context of the image
 - Follows typical architecture of a CNN:
 - Repeated application of 2 unpadded 3x3 convolutions
 - Each followed by ReLU activation
 - 2x2 maxpooling operation with stride 2 for downsampling
 - At each downsampling step, # of channels is doubled
- as before: Height, Width ↓, Depth: ↑

U-Net: Decoder

Right Side: **Expansion Path** (Decoder):

- Upsampling to recover spatial locations for assigning class labels to each pixel
 - 2x2 up-convolution that halves number of input channels
 - **Skip Connections**: outputs of up-convolutions are concatenated with feature maps from encoder
 - Followed by 2 ordinary 3x3 convs
 - final layer: 1x1 conv to map 64 channels to # classes
- Height, Width:  , Depth: 